

Load dataset into a Delta table

09:07 AM (16s) 3

```
df=spark.read.csv("/Workspace/Users/srithankodumuru22@gmail.com/Drafts/Data/Sample - Superstore.csv",
header=True,
inferSchema=True
)
```

df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: integer, Order ID: string ... 19 more fields]

09:08 AM (14s) 4 Python

```
display(df)
```

[See performance \(1\)](#) ✓ Checked for improvements

Table +

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Clai
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Clai
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Dar
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sea
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sea
6	6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
7	7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
8	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro

09:10 AM (10s) 6

```
df.write.format("delta").option("delta.columnMapping.mode", "name")\
.mode("overwrite")\
.saveAsTable("customer_master")
```

[See performance \(1\)](#) ✓ Checked for improvements

09:11 AM (4s) 7

```
%sql
SELECT * FROM customer_master;
```

[See performance \(1\)](#) ✓ Checked for improvements

Table +

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Clai
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Clai
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Dar
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sea
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sea
6	6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
7	7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
8	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro

Data Cleaning

09:16 AM (2s) 9 Python

```
df_clean = df.dropna()
display(df_clean)
```

See performance (1) ✓ Checked for improvements

df_clean: pyspark.sql.connect.dataframe.DataFrame = [Row ID: integer, Order ID: string ... 19 more fields]

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	
1	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Clai
2	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Clai
3	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Dar
4	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sea
5	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sea
6	6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
7	7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
8	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
9	9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
10	10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
11	11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
12	12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Bro
13	13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Anc
14	14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irer

09:16 AM (2s) 10 Python

```
df_clean = df_clean.dropDuplicates()
display(df_clean)
```

See performance (1) ✓ Checked for improvements

df_clean: pyspark.sql.connect.dataframe.DataFrame = [Row ID: integer, Order ID: string ... 19 more fields]

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	
1	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sea
2	17	CA-2014-105893	2014-11-11	2014-11-18	Standard Class	PK-19075	Petv
3	21	CA-2014-143336	2014-08-27	2014-09-01	Second Class	ZD-21925	Zus
4	53	CA-2015-115742	2015-04-18	2015-04-22	Standard Class	DP-13000	Dar
5	118	CA-2015-110457	2015-03-02	2015-03-06	Standard Class	DK-13090	Dav
6	140	CA-2016-145583	2016-10-13	2016-10-19	Standard Class	LC-16885	Len
7	145	CA-2017-155376	2017-12-22	2017-12-27	Standard Class	SG-20080	San
8	147	CA-2014-110072	2014-10-22	2014-10-28	Standard Class	MG-17680	Ma
9	186	CA-2016-105018	2016-11-28	2016-12-02	Standard Class	SK-19990	Sall
10	200	US-2017-124303	2017-07-06	2017-07-13	Standard Class	FH-14365	Fre
11	237	CA-2017-160514	2017-11-12	2017-11-16	Standard Class	DB-13120	Dav
12	246	CA-2014-131926	2014-06-01	2014-06-06	Second Class	DW-13480	Dia
13	260	CA-2017-163139	2017-12-01	2017-12-03	Second Class	CC-12670	Cra
14	263	US-2014-106992	2014-09-19	2014-09-21	Second Class	SB-20290	Sea

The dataset contained 9,994 records. After applying `dropna()` and `dropDuplicates()`, the row count remained 9,994. This indicates that there were no missing values or duplicate records to remove.

Create an Incremental Dataset

The screenshot shows a Databricks notebook cell with the following Python code:

```
incremental_data= [(9995,"CG-12520","Consumer","United States","California","Los Angeles","West",500),  
(2,"DV-13045","Corporate","United States","Texas","Dallas","Central",800)]  
columns=["Row ID","Customer ID","Segment","Country","State","City","Region","Sales"]  
incremental_df =spark.createDataFrame([incremental_data,columns])  
display(incremental_df)
```

Below the code, the table view shows the following data:

Row ID	Customer ID	Segment	Country	State	City	Region	Sales
1	9995	CG-12520	Consumer	United States	California	Los Angeles	West
2	2	DV-13045	Corporate	United States	Texas	Dallas	Central

The interface also shows "2 rows | 1.058s runtime" and "Refreshed 16 minutes ago".

The screenshot shows a Databricks notebook cell with the following Python code:

```
incremental_df.write.format("delta") \  
.option("delta.columnMapping.mode", "name") \  
.mode("overwrite") \  
.saveAsTable("customer_incremental")
```

Below the code, the table view shows the same data as the previous screenshot:

Row ID	Customer ID	Segment	Country	State	City	Region	Sales
1	9995	CG-12520	Consumer	United States	California	Los Angeles	West
2	2	DV-13045	Corporate	United States	Texas	Dallas	Central

The interface also shows "09:23 AM (2s)" and "Refreshed 16 minutes ago".

Apply MERGE operation

▶

✓ 09:27 AM (10s)

17

SQL

🗑️

🔗

📄

⋮

```

%sql
MERGE INTO customer_master AS target
USING customer_incremental AS source
ON target.`Row ID`=source.`Row ID`
WHEN MATCHED THEN
UPDATE SET
  target.`Customer ID`=source.`Customer ID`,
  target.`Segment`=source.`Segment`,
  target.`Country`=source.`Country`,
  target.`State`=source.`State`,
  target.`City`= source.`City`,
  target.`Region` = source.`Region`,
  target.`Sales`=source.`Sales`

WHEN NOT MATCHED THEN
INSERT(`Row ID`,`Customer ID`,`Segment`,`Country`,`State`,`City`,`Region`,`Sales`)
VALUES(source.`Row ID`,source.`Customer ID`,source.`Segment`, source.`Country`,source.`State`, source.`City`,
source.`Region`, source.`Sales`)

```

[See performance \(1\)](#)

Optimize

>

📄

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_updated_rows: long ... 2 more fields]

Table ▾

+

🔍

🏠

📄

📄

	1 2 3 num_affected_rows	1 2 3 num_updated_rows	1 2 3 num_deleted_...	1 2 3 num_inserted_rows
1	2	1	num_deleted_rows	1

The MERGE operation was successful. One existing record was updated, and one new record was added to the Delta table. No records were deleted. In total, 2 records were changed.